ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA TOÁN - ỨNG DỤNG

——oOo——



BÁO CÁO LAB 02
DEEP LEARNING

ĐỀ TÀI: DỰ ĐOÁN GIÁ NHÀ BẰNG MẠNG MLP TRÊN BỘ DỮ LIỆU HOUSE PRICES

GIẢNG VIÊN HƯỚNG DẪN: TS. ĐỖ NHƯ TÀI

THÀNH VIÊN: 3123580020 - NGUYỄN QUỐC KỲ

3123580037 - NGUYỄN ĐỨC QUANG

3123580027 - NGUYỄN HIẾU NGÂN

LỚP: KHOA HỌC DỮ LIỆU - DDU1231

TP. Hồ Chí Minh, tháng 09, năm 2026

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA TOÁN - ỨNG DỤNG

——oOo——

BÁO CÁO LAB 02
DEEP LEARNING

ĐỀ TÀI: DỰ ĐOÁN GIÁ NHÀ BẰNG MẠNG MLP TRÊN BỘ DỮ LIỆU HOUSE PRICES

GIẢNG VIÊN HƯỚNG DẪN: TS. ĐỖ NHƯ TÀI

THÀNH VIÊN: 3123580020 - NGUYỄN QUỐC KỲ

3123580037 - NGUYỄN ĐỨC QUANG

3123580027 - NGUYỄN HIẾU NGÂN

LỚP: KHOA HỌC DỮ LIỆU - DDU1231

TP. Hồ Chí Minh, tháng 09, năm 2026

BẢNG ĐÁNH GIÁ THÀNH VIÊN

STT	MSSV	Họ và tên	Nhiệm vụ	Đóng góp	Chữ ký
1	3123580020	Nguyễn Quốc Kỳ	Phân tích dữ liệu, thực hiện EDA và trực quan hóa; phụ trách Giới thiệu, Business Understanding, Data Understanding, tổng kết và hoàn thiện báo cáo.	100%	
2	3123580037	Nguyễn Đức Quang	Xây dựng quy trình tiền xử lý dữ liệu, xử lý missing values, encoding, scaling, kiểm soát data leakage và Ridge Regression baseline; phụ trách Data Preparation và phần baseline trong Modeling.	100%	
3	3123580027	Nguyễn Hiếu Ngân	Xây dựng và huấn luyện MLP bằng PyTorch, Early Stopping, thử nghiệm kiến trúc, kiểm tra stability qua nhiều seed và đánh giá mô hình; phụ trách Modeling MLP và Evaluation.	100%	

TP. Hồ Chí Minh, ngày ..... tháng 09 năm 2026

Trưởng nhóm xác nhận

LỜI CẢM ƠN

Nhóm chúng em xin chân thành cảm ơn TS. Đỗ Như Tài đã hướng dẫn và cung cấp kiến thức trong học phần Deep Learning. Những nội dung về mạng neural, quá trình huấn luyện mô hình, đánh giá kết quả và thực hành với PyTorch đã giúp nhóm có cơ sở để hoàn thành bài Lab 02 về dự đoán giá nhà.

Trong quá trình thực hiện, nhóm đã có cơ hội vận dụng kiến thức về xử lý dữ liệu dạng bảng, bài toán hồi quy, mạng Multi-Layer Perceptron và quy trình CRISP-DM vào một bộ dữ liệu thực tế. Đây là cơ hội giúp nhóm hiểu rõ hơn không chỉ cách xây dựng mô hình mà còn cách kiểm soát data leakage, đánh giá tính ổn định và phân tích kết quả một cách khách quan.

Nhóm xin chân thành cảm ơn thầy!

TP. Hồ Chí Minh, tháng 09, năm 2026

Nhóm sinh viên thực hiện:
NGUYỄN QUỐC KỲ
NGUYỄN ĐỨC QUANG
NGUYỄN HIẾU NGÂN

MỤC LỤC

1 GIỚI THIỆU ĐỀ TÀI **1**
- 1.1 Tổng quan đề tài . . . 1
- 1.2 Quy trình thực hiện . . . 1

2 BUSINESS UNDERSTANDING **1**
- 2.1 Bối cảnh và bài toán . . . 2
- 2.2 Mục tiêu của bài thực hành . . . 2
- 2.3 Mục tiêu Deep Learning . . . 2
- 2.4 Tiêu chí đánh giá . . . 2

3 DATA UNDERSTANDING **3**
- 3.1 Tổng quan về bộ dữ liệu . . . 4
- 3.2 Khảo sát biến mục tiêu SalePrice . . . 4
- 3.3 Phân tích dữ liệu thiếu . . . 4
- 3.4 Phân tích mối tương quan với SalePrice . . . 5
- 3.5 Khảo sát một số thuộc tính phân loại . . . 6

4 DATA PREPARATION **6**
- 4.1 Nguyên tắc tiền xử lý . . . 7
- 4.2 Xử lý dữ liệu thiếu . . . 7
- 4.3 Mã hóa biến có thứ tự . . . 7
- 4.4 Mã hóa biến danh nghĩa . . . 8
- 4.5 Chuẩn hóa dữ liệu . . . 8
- 4.6 Biến đổi biến mục tiêu . . . 8
- 4.7 Kết quả sau tiền xử lý . . . 8

5 MODELING **9**
- 5.1 Ridge Regression baseline . . . 10
- 5.2 Chuẩn bị dữ liệu cho PyTorch . . . 10
- 5.3 Kiến trúc mạng MLP . . . 10
- 5.4 Cấu hình huấn luyện . . . 11
- 5.5 Thử nghiệm nhiều kiến trúc . . . 11

6 EVALUATION **11**
- 6.1 So sánh Ridge và MLP baseline . . . 12
- 6.2 Đường cong huấn luyện . . . 12
- 6.3 Phân tích dự đoán và residual . . . 13
- 6.4 So sánh các kiến trúc MLP . . . 13
- 6.5 Kiểm tra độ ổn định qua nhiều seed . . . 14
- 6.6 So sánh mô hình MLP cuối với Ridge Regression . . . 15

6.7 Kết luận giai đoạn Evaluation 15

7 DEPLOYMENT VÀ TỔNG KẾT **15**

7.1 Kết quả đạt được 16

7.2 Nhận xét về kết quả mô hình 16

7.3 Hạn chế và hướng phát triển 17

TÀI LIỆU THAM KHẢO **18**

LỜI MỞ ĐẦU

Trong những năm gần đây, học sâu đã được ứng dụng rộng rãi trong nhiều bài toán dự đoán và phân tích dữ liệu. Bên cạnh các bài toán xử lý hình ảnh hoặc ngôn ngữ tự nhiên, mạng neural cũng có thể được áp dụng cho dữ liệu dạng bảng nhằm học các mối quan hệ giữa nhiều đặc trưng đầu vào và một biến mục tiêu.

Trong Lab 02 của học phần Deep Learning, nhóm thực hiện bài toán dự đoán giá nhà trên bộ dữ liệu *House Prices: Advanced Regression Techniques*. Mục tiêu chính là xây dựng mạng Multi-Layer Perceptron bằng PyTorch, thực hiện tiền xử lý dữ liệu phù hợp, đánh giá mô hình bằng các chỉ số hồi quy và so sánh kết quả với một mô hình baseline. Toàn bộ quá trình được tổ chức theo quy trình CRISP-DM nhằm đảm bảo tính hệ thống và khả năng tái lập.

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1 Tổng quan đề tài

Trong lĩnh vực bất động sản, việc xác định giá bán của một căn nhà phụ thuộc vào nhiều yếu tố khác nhau như vị trí, diện tích, chất lượng xây dựng, số phòng, năm xây dựng, tầng hầm, garage và các tiện ích đi kèm. Do số lượng yếu tố ảnh hưởng tương đối lớn và mối quan hệ giữa các yếu tố này với giá bán không phải lúc nào cũng tuyến tính, bài toán dự đoán giá nhà là một bài toán phù hợp để áp dụng các phương pháp học máy và học sâu.

Trong bài thực hành này, nhóm thực hiện bài toán House Prices: Advanced Regression Techniques, với mục tiêu xây dựng mô hình dự đoán giá bán cuối cùng của một căn nhà thông qua các thuộc tính mô tả đặc điểm của căn nhà. Biến cần dự đoán là SalePrice, đây là một đại lượng liên tục nên bài toán được xác định là bài toán hồi quy có giám sát (Supervised Learning – Regression).

Mô hình chính được sử dụng trong bài là Multi-Layer Perceptron (MLP) được xây dựng bằng thư viện PyTorch. Mạng MLP được lựa chọn nhằm áp dụng kiến thức về mạng neural nhiều lớp vào dữ liệu dạng bảng, đồng thời khảo sát khả năng học mối quan hệ giữa nhiều đặc trưng đầu vào và giá bán nhà. Bên cạnh mô hình MLP, bài thực hành còn sử dụng Ridge Regression làm mô hình cơ sở để tạo mốc so sánh với mô hình học sâu.

Do phân bố của SalePrice có độ lệch phải, biến mục tiêu được biến đổi bằng hàm log1p. Vì vậy, chỉ số đánh giá chính của bài là RMSE trên log1p(SalePrice). Ngoài RMSE, bài còn sử dụng thêm MAE để biểu diễn mức sai lệch trung bình theo đơn vị giá thực tế và R^2 để đánh giá mức độ mô hình giải thích được sự biến thiên của giá nhà.

Toàn bộ quá trình thực hiện được tổ chức theo quy trình CRISP-DM, bao gồm sáu giai đoạn chính: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation và Deployment. Quy trình này giúp bài thực hành được triển khai có hệ thống, từ việc xác định mục tiêu bài toán đến xây dựng, đánh giá và sử dụng mô hình để tạo kết quả dự đoán cuối cùng.

1.2 Quy trình thực hiện

Toàn bộ bài thực hành được tổ chức theo quy trình CRISP-DM gồm các giai đoạn: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation và Deployment. Quy trình này giúp nhóm triển khai bài toán theo một trình tự rõ ràng, từ xác định mục tiêu đến tạo file dự đoán cuối cùng.

CHƯƠNG 2: BUSINESS UNDERSTANDING

2.1 Bối cảnh và bài toán

Giá bán của một căn nhà được quyết định bởi nhiều thuộc tính khác nhau. Một số thuộc tính phản ánh trực tiếp quy mô của căn nhà như diện tích sử dụng, diện tích tầng hầm hay số phòng; trong khi các thuộc tính khác thể hiện chất lượng và điều kiện sử dụng như chất lượng xây dựng, năm xây dựng, năm cải tạo, tình trạng garage, fireplace hoặc khu dân cư.

Trong bộ dữ liệu House Prices, mỗi căn nhà được mô tả bởi nhiều nhóm thuộc tính khác nhau, bao gồm vị trí và khu dân cư, diện tích, chất lượng xây dựng, năm xây dựng hoặc cải tạo, tầng hầm, garage, fireplace, số phòng và các đặc điểm liên quan khác. Mục tiêu đặt ra là sử dụng các đặc trưng này để dự đoán chính xác giá bán cuối cùng SalePrice.

Bài toán được xác định là bài toán hồi quy, bởi vì giá bán nhà là một biến số liên tục. Khác với bài toán phân loại, đầu ra của mô hình không phải là một nhãn cố định mà là một giá trị số biểu diễn mức giá dự kiến của căn nhà.

2.2 Mục tiêu của bài thực hành

Ngoài mục tiêu xây dựng được mô hình có khả năng dự đoán giá nhà, bài thực hành còn tập trung xây dựng một quy trình xử lý dữ liệu phù hợp với dữ liệu dạng bảng, trong đó dữ liệu thiếu được xử lý dựa trên ý nghĩa của từng thuộc tính, các biến phân loại được mã hóa theo đúng bản chất dữ liệu và các đặc trưng được chuẩn hóa trước khi đưa vào mạng neural. Bên cạnh đó, bài thực hành tiến hành xây dựng và huấn luyện mô hình MLP bằng PyTorch, đánh giá khả năng tổng quát hóa của mô hình trên tập validation, đồng thời so sánh nhiều kiến trúc MLP khác nhau và kiểm tra mức độ ổn định của mô hình khi thay đổi seed. Kết quả của MLP cũng được đối chiếu với mô hình baseline Ridge Regression để có cơ sở đánh giá khách quan hơn. Cuối cùng, mô hình được lựa chọn được sử dụng để dự đoán giá nhà trên tập test và tạo file `submission.csv`.

2.3 Mục tiêu Deep Learning

Trọng tâm của Lab là xây dựng mạng MLP bằng PyTorch cho bài toán hồi quy. Bên cạnh việc huấn luyện một mô hình duy nhất, bài thực hành tiến hành so sánh ba kiến trúc MLP khác nhau gồm **64–32–16**, **128–64–32** và **256–128–64**. Việc thử nghiệm nhiều kiến trúc giúp đánh giá ảnh hưởng của số lượng neuron trong các lớp ẩn đến hiệu quả dự đoán, từ đó lựa chọn cấu hình phù hợp hơn với bộ dữ liệu.

Kiến trúc được chọn tiếp tục được chạy với các seed 42, 123 và 2026 để đánh giá tính ổn định của mô hình trước ảnh hưởng của khởi tạo trọng số và thứ tự batch.

2.4 Tiêu chí đánh giá

Chỉ số chính được sử dụng là RMSE trên log scale:

$$RMSE_{\log} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(\log(1+y_i) - \hat{z}_i\right)^2}$$

Trong đó y_i là giá bán thực tế và $\hat{z}_i$ là giá trị log-price do mô hình dự đoán.

Ngoài RMSE log, hai chỉ số bổ sung được sử dụng gồm:

MAE – Mean Absolute Error: cho biết mức sai lệch tuyệt đối trung bình giữa giá

dự đoán và giá thực tế. Vì MAE được tính sau khi chuyển ngược từ log scale về giá thực tế, chỉ số này dễ diễn giải theo đơn vị tiền tệ.

R^2 – Coefficient of Determination: phản ánh khả năng mô hình giải thích sự biến thiên của giá bán. Giá trị R^2 càng gần 1 cho thấy mô hình càng giải thích tốt sự khác biệt giữa các mức giá trong dữ liệu.

CHƯƠNG 3: DATA UNDERSTANDING

3.1 Tổng quan về bộ dữ liệu

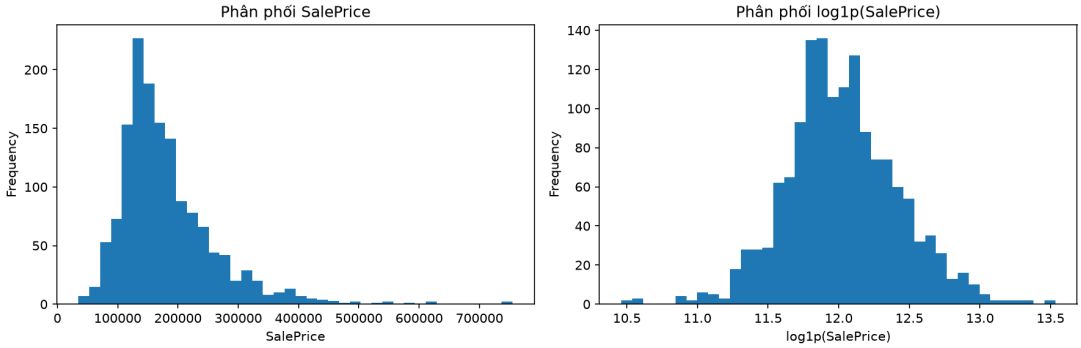
Bộ dữ liệu được sử dụng trong bài toán gồm hai tập chính là train.csv và test.csv. Tập huấn luyện có 1.460 dòng và 81 cột, trong đó bao gồm các thuộc tính mô tả căn nhà và biến mục tiêu SalePrice. Tập kiểm thử có 1.459 dòng và 80 cột, có cấu trúc tương tự tập huấn luyện nhưng không chứa biến SalePrice.

Các thuộc tính trong dữ liệu có nhiều kiểu khác nhau. Trong tập train có 43 biến kiểu object, 35 biến kiểu số nguyên (int64) và 3 biến kiểu số thực (float64). Điều này cho thấy bộ dữ liệu House Prices là dữ liệu dạng bảng hỗn hợp, bao gồm cả biến số và biến phân loại. Do đó, trước khi đưa dữ liệu vào mô hình MLP, cần thực hiện các bước mã hóa và chuẩn hóa phù hợp cho từng nhóm thuộc tính.

3.2 Khảo sát biến mục tiêu SalePrice

Biến SalePrice biểu diễn giá bán cuối cùng của mỗi căn nhà và là biến mục tiêu của bài toán. Qua thống kê mô tả trên 1.460 mẫu, giá bán trung bình của các căn nhà vào khoảng 180.921 USD, trong khi giá trị trung vị là 163.000 USD. Giá thấp nhất trong tập dữ liệu là 34.900 USD và giá cao nhất đạt 755.000 USD. Khoảng 50% số căn nhà có giá nằm trong khoảng từ 129.975 USD đến 214.000 USD.

Sự chênh lệch giữa giá trị trung bình và trung vị cho thấy phân bố của SalePrice không hoàn toàn cân đối. Quan sát biểu đồ phân phối cho thấy phần lớn các căn nhà tập trung ở vùng giá thấp và trung bình, trong khi một số ít căn có giá rất cao làm phân bố kéo dài về phía bên phải.



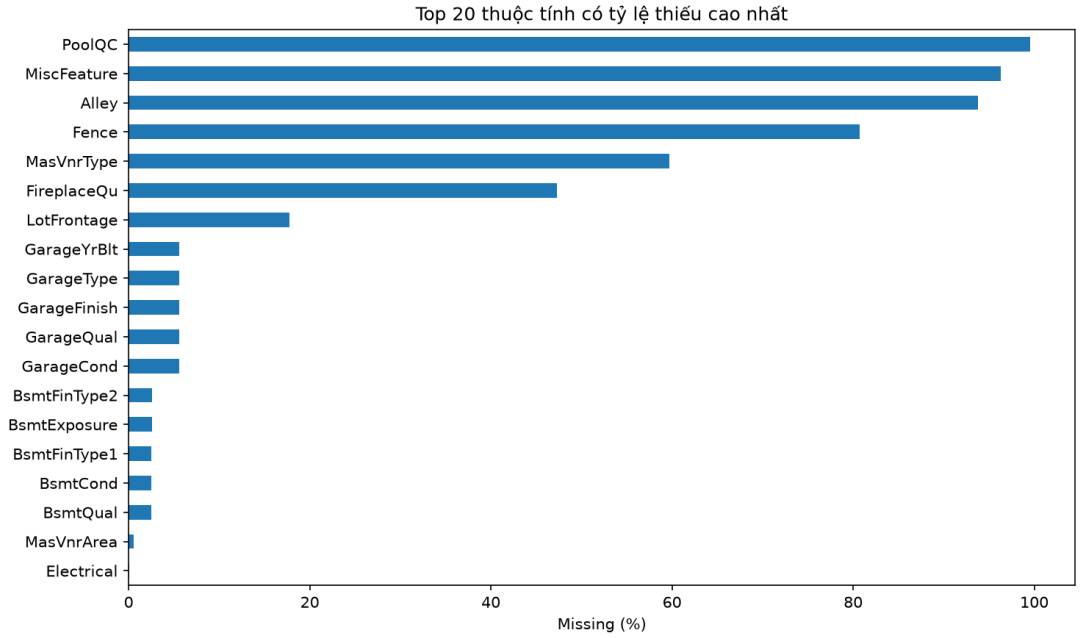
Hình 3.1: Phân phối SalePrice trước và sau phép biến đổi log1p

Độ lệch của SalePrice ban đầu là khoảng 1,883, cho thấy phân bố lệch phải khá rõ. Sau khi áp dụng phép biến đổi log1p, độ lệch giảm xuống còn khoảng 0,121, giúp phân bố trở nên cân đối hơn. Vì vậy, việc sử dụng log1p(SalePrice) giúp giảm ảnh hưởng của các giá trị lớn, đồng thời tạo điều kiện thuận lợi hơn cho quá trình học của mô hình MLP.

3.3 Phân tích dữ liệu thiếu

Qua quá trình khảo sát dữ liệu, nhóm nhận thấy dữ liệu chứa một lượng đáng kể các giá trị thiếu ở nhiều thuộc tính. Một số cột có tỷ lệ thiếu rất cao, tiêu biểu như `PoolQC` với khoảng 99,52%, `MiscFeature` khoảng 96,30%, `Alley` khoảng 93,77%, `Fence` khoảng 80,75%, `MasVnrType` khoảng 59,73% và `FireplaceQu` khoảng 47,26%. Bên cạnh đó, thuộc tính `LotFrontage` có khoảng 17,74% giá trị thiếu, trong khi các thuộc tính liên quan đến garage và basement có tỷ lệ thiếu thấp hơn. Kết quả này cho thấy bước xử lý

giá trị thiếu là cần thiết trước khi đưa dữ liệu vào quá trình huấn luyện mô hình.



Hình 3.2: Các thuộc tính có số lượng giá trị thiếu lớn nhất

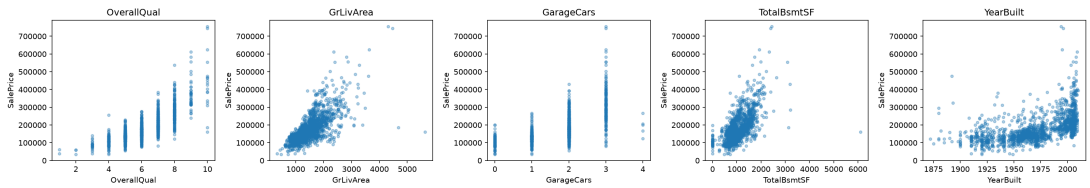
Tuy nhiên, không phải mọi giá trị thiếu đều mang ý nghĩa là dữ liệu bị mất. Đối với nhiều thuộc tính như hồ bơi, hàng rào, garage, tầng hầm hoặc fireplace, giá trị thiếu có thể phản ánh việc căn nhà không có tiện ích tương ứng. Vì vậy, trong giai đoạn Data Preparation, các giá trị thiếu sẽ được xử lý dựa trên ý nghĩa cụ thể của từng thuộc tính thay vì áp dụng một phương pháp điền dữ liệu duy nhất cho toàn bộ tập dữ liệu.

3.4 Phân tích mối tương quan với SalePrice

Đối với các biến số, hệ số tương quan được sử dụng để khảo sát mức độ liên hệ với SalePrice. Kết quả cho thấy một số đặc trưng có tương quan dương tương đối mạnh với giá bán.

Trong đó, OverallQual có hệ số tương quan cao nhất, đạt khoảng 0,7910, tiếp theo là GrLivArea với 0,7086, GarageCars với 0,6404, GarageArea với 0,6234, TotalBsmtSF với 0,6136 và 1stFlrSF với 0,6059. Một số thuộc tính khác như FullBath, TotRmsAbvGrd, YearBuilt và YearRemodAdd cũng thể hiện mức tương quan đáng kể với giá bán.

Để quan sát rõ hơn mối quan hệ giữa các đặc trưng đầu vào và SalePrice, bài thực hành lựa chọn một số biến tiêu biểu gồm OverallQual, GrLivArea, GarageCars, TotalBsmtSF và YearBuilt.



Hình 3.3: Mối quan hệ giữa một số thuộc tính số và SalePrice

Qua biểu đồ có thể nhận thấy rằng khi chất lượng tổng thể (OverallQual) tăng thì giá bán thường tăng rõ rệt. Tương tự, các căn nhà có diện tích sử dụng lớn hơn (GrLivArea),

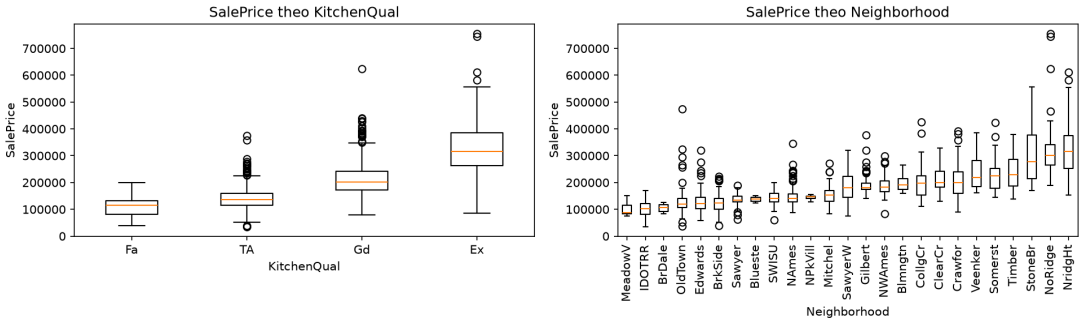
garage có sức chứa lớn hơn hoặc tầng hầm có diện tích lớn thường có mức giá cao hơn. Đối với YearBuilt, các căn nhà được xây dựng gần đây nhìn chung có xu hướng đạt mức giá cao hơn.

3.5 Khảo sát một số thuộc tính phân loại

Bên cạnh các biến số, bộ dữ liệu House Prices còn chứa nhiều biến phân loại có thể ảnh hưởng đáng kể đến giá bán. Trong phần này, hai thuộc tính tiêu biểu được lựa chọn để khảo sát là KitchenQual và Neighborhood. Đây là hai biến đại diện cho hai nhóm thông tin khác nhau: KitchenQual phản ánh chất lượng nhà bếp, trong khi Neighborhood thể hiện khu vực dân cư nơi căn nhà tọa lạc. Notebook trực tiếp xây dựng hai boxplot để quan sát SalePrice theo hai thuộc tính này.

Đối với KitchenQual, các mức chất lượng được sắp xếp theo thứ tự từ thấp đến cao gồm Po, Fa, TA, Gd và Ex. Việc biểu diễn các mức này theo thứ tự giúp quan sát rõ xu hướng thay đổi của giá bán khi chất lượng nhà bếp được cải thiện. Qua biểu đồ có thể nhận thấy các nhóm có chất lượng nhà bếp cao hơn thường đi kèm với mức SalePrice cao hơn. Điều này cho thấy KitchenQual là một biến có ý nghĩa đối với bài toán dự đoán giá nhà và cần được giữ lại trong quá trình xây dựng mô hình. Trong giai đoạn tiền xử lý, thuộc tính này cũng được xem là biến có thứ tự và được ánh xạ theo thang chất lượng thay vì xử lý như một biến danh nghĩa thông thường.

Đối với Neighborhood, dữ liệu được nhóm theo từng khu dân cư và sắp xếp theo trung vị của SalePrice. Cách trình bày này giúp thấy rõ sự khác biệt về giá nhà giữa các khu vực khác nhau. Một số neighborhood có mức giá trung vị cao hơn rõ rệt so với các khu vực còn lại, trong khi một số khu vực tập trung ở nhóm giá thấp hơn. Điều đó cho thấy vị trí địa lý là một yếu tố quan trọng có liên quan đến giá bán của căn nhà.



Hình 3.4: Phân bố SalePrice theo một số thuộc tính phân loại

Từ Hình 3.4 có thể nhận thấy cả chất lượng nhà bếp và khu dân cư đều tạo ra sự khác biệt đáng kể trong phân bố giá bán. KitchenQual thể hiện một xu hướng tương đối rõ theo cấp độ chất lượng, vì vậy thuộc tính này phù hợp với cách mã hóa ordinal. Ngược lại, Neighborhood không có một thứ tự tự nhiên giữa các giá trị, nên ở bước Data Preparation sẽ được xử lý theo hướng mã hóa biến phân loại danh nghĩa.

Nhìn chung, kết quả khảo sát các biến phân loại cho thấy việc chỉ sử dụng các biến số là chưa đủ để mô tả đầy đủ giá bán của căn nhà. Các đặc trưng phân loại chứa nhiều thông tin có giá trị và cần được xử lý phù hợp trước khi đưa vào mô hình. Đây cũng là cơ sở cho việc áp dụng đồng thời Ordinal Encoding và One-Hot Encoding trong giai đoạn tiền xử lý dữ liệu.

CHƯƠNG 4: DATA PREPARATION

4.1 Nguyên tắc tiền xử lý

Sau giai đoạn khám phá dữ liệu, dữ liệu được đưa qua bước tiền xử lý nhằm chuyển đổi các thuộc tính về dạng phù hợp trước khi huấn luyện mô hình. Một yêu cầu quan trọng trong quá trình này là hạn chế hiện tượng data leakage, tức là thông tin từ tập validation hoặc tập test ảnh hưởng đến quá trình học của mô hình.

Vì vậy, trong bài thực hành, tập train.csv được tách thành tập huấn luyện và tập validation trước khi thực hiện các bước tiền xử lý có học tham số từ dữ liệu. Các giá trị như median, mode, tham số của bộ mã hóa và tham số chuẩn hóa chỉ được xác định trên tập huấn luyện. Sau đó, các tham số đã học này mới được sử dụng để biến đổi tập validation và tập test.

Sau khi loại bỏ Id và tách biến mục tiêu SalePrice, dữ liệu huấn luyện ban đầu được chia thành 1.168 mẫu dùng để huấn luyện và 292 mẫu dùng để validation, tương ứng với tỷ lệ 80% – 20%. Tập test Kaggle gồm 1.459 mẫu được giữ riêng và không tham gia vào quá trình lựa chọn mô hình.

Cách tổ chức này giúp kết quả trên tập validation phản ánh khách quan hơn khả năng dự đoán của mô hình đối với dữ liệu chưa từng được sử dụng trong quá trình huấn luyện.

4.2 Xử lý dữ liệu thiếu

Qua bước Data Understanding, có thể thấy bộ dữ liệu chứa nhiều giá trị thiếu, tuy nhiên ý nghĩa của các giá trị thiếu không giống nhau. Vì vậy, bài thực hành không sử dụng một phương pháp điền dữ liệu duy nhất cho tất cả các thuộc tính mà xử lý dựa trên ý nghĩa của từng nhóm biến.

Đối với các biến phân loại mà giá trị thiếu thể hiện việc căn nhà không có tiện ích tương ứng, chẳng hạn như GarageType, BsmtQual, FireplaceQu, PoolQC, Fence hoặc Alley, giá trị thiếu được thay bằng nhãn "None". Cách xử lý này giúp bảo toàn ý nghĩa thực tế của dữ liệu thay vì xem các trường hợp này là dữ liệu bị mất hoàn toàn.

Đối với các biến số liên quan đến những tiện ích không tồn tại, chẳng hạn như GarageArea, GarageCars, TotalBsmtSF, BsmtFinSF1, BsmtFinSF2 hoặc MasVnrArea, giá trị thiếu được thay bằng 0. Giá trị 0 trong trường hợp này thể hiện rằng căn nhà không có diện tích hoặc số lượng tương ứng với tiện ích đó.

Riêng thuộc tính LotFrontage, biểu diễn chiều dài mặt tiền của lô đất, được xử lý bằng trung vị theo từng Neighborhood. Cách xử lý này được lựa chọn vì các căn nhà trong cùng khu vực có thể có đặc điểm mặt tiền tương đồng hơn so với toàn bộ dữ liệu. Nếu xuất hiện một khu vực không có giá trị trung vị phù hợp, trung vị chung của tập huấn luyện được sử dụng thay thế.

Đối với các biến phân loại còn thiếu sau các bước trên, giá trị thiếu được điền bằng mode của thuộc tính tương ứng trên tập huấn luyện. Các biến số còn thiếu được điền bằng median của tập huấn luyện. Nhờ đó, toàn bộ các giá trị sử dụng để bù dữ liệu đều được xác định từ training fold và không sử dụng thông tin từ validation hoặc test.

4.3 Mã hóa biến có thứ tự

Những biến có thứ tự tự nhiên được xử lý bằng Ordinal Encoding. Ví dụ, nhóm mức chất lượng `Ex > Gd > TA > Fa > Po > None` được ánh xạ thành các giá trị số

theo thứ tự. Cách xử lý tương tự được áp dụng cho `BsmtExposure`, `BsmtFinType1`, `BsmtFinType2`, `GarageFinish`, `Fence`, `PavedDrive` và `CentralAir`.

4.4 Mã hóa biến danh nghĩa

Các biến phân loại không có thứ tự tự nhiên như khu dân cư, loại mái nhà, kiểu công trình hoặc điều kiện bán được xử lý bằng One-Hot Encoding. Mỗi giá trị phân loại được biểu diễn bằng một biến nhị phân riêng, giúp chuyển dữ liệu từ dạng chuỗi sang dạng số phù hợp với mô hình.

Trong quá trình này, bộ mã hóa được thiết lập để có thể xử lý các giá trị phân loại mới xuất hiện trong tập validation hoặc tập test mà không làm gián đoạn quá trình dự đoán. Nhờ đó, dữ liệu sau mã hóa giữa train, validation và test luôn được duy trì cùng cấu trúc đầu vào.

4.5 Chuẩn hóa dữ liệu

Sau khi xử lý dữ liệu thiếu và mã hóa biến phân loại, các biến số và các biến ordinal được chuẩn hóa bằng StandardScaler. Quá trình chuẩn hóa đưa các thuộc tính về thang đo tương đối đồng nhất, hạn chế trường hợp một biến có giá trị lớn chi phối quá trình cập nhật trọng số của mạng neural.

Các biến được tạo ra từ One-Hot Encoding vẫn được giữ dưới dạng giá trị 0 và 1. Điều này giúp tránh thay đổi ý nghĩa của các biến nhị phân trong quá trình chuẩn hóa.

Việc chuẩn hóa đặc biệt cần thiết đối với MLP vì các đặc trưng đầu vào của bộ dữ liệu House Prices có thang đo rất khác nhau. Chẳng hạn, một số thuộc tính biểu diễn diện tích có thể có giá trị hàng nghìn, trong khi các thuộc tính chất lượng chỉ nằm trong một số mức nhỏ.

4.6 Biến đổi biến mục tiêu

Biến mục tiêu SalePrice được biến đổi bằng log1p trước khi huấn luyện. Như đã phân tích ở phần Data Understanding, phép biến đổi này làm giảm đáng kể độ lệch phải của phân bố giá nhà và hạn chế ảnh hưởng của các căn nhà có giá trị quá cao.

Đối với mô hình MLP, biến mục tiêu sau phép biến đổi log tiếp tục được chuẩn hóa dựa trên giá trị trung bình và độ lệch chuẩn của training fold. Trong lần thực nghiệm hiện tại, giá trị trung bình của log-target trên tập huấn luyện là khoảng 12,0307, với độ lệch chuẩn khoảng 0,3904.

Việc chuẩn hóa target giúp giá trị đầu ra mà mạng cần học nằm trong khoảng ổn định hơn, từ đó hỗ trợ quá trình tối ưu gradient. Khi đánh giá mô hình, kết quả dự đoán được chuyển ngược về log scale ban đầu trước khi tính RMSE, đồng thời được chuyển tiếp về đơn vị giá thực tế để tính MAE và R^2.

4.7 Kết quả sau tiền xử lý

Sau khi hoàn thành toàn bộ quá trình tiền xử lý, dữ liệu đầu vào đã được chuyển hoàn toàn sang dạng số và số lượng đặc trưng tăng lên do quá trình One-Hot Encoding.

Bảng 4.1: Kích thước dữ liệu sau tiền xử lý ở bước validation experiment

Tập dữ liệu	**Số mẫu**	**Số đặc trưng**
Training	1.168	248
Validation	292	248
Test	1.459	248

Cả ba tập dữ liệu đều có cùng 248 đặc trưng, đảm bảo cấu trúc đầu vào nhất quán cho quá trình huấn luyện và dự đoán. Quá trình kiểm tra sau tiền xử lý cũng cho thấy dữ liệu không còn giá trị NaN hoặc vô hạn (Inf).

Như vậy, sau giai đoạn Data Preparation, dữ liệu đã đáp ứng các yêu cầu cần thiết để đưa vào các mô hình hồi quy. Đây là cơ sở để chuyển sang bước tiếp theo là Modeling, trong đó Ridge Regression được sử dụng làm baseline và mạng MLP bằng PyTorch được xây dựng làm mô hình chính.

CHƯƠNG 5: MODELING

5.1 Ridge Regression baseline

Trước khi xây dựng mạng neural, bài thực hành sử dụng Ridge Regression làm mô hình baseline. Mục đích của baseline là tạo ra một mốc tham chiếu để đánh giá xem mô hình MLP có thực sự cải thiện khả năng dự đoán hay không. Ridge được huấn luyện trên cùng tập dữ liệu đã qua tiền xử lý và được đánh giá trên cùng tập validation với MLP, nhờ đó việc so sánh giữa hai mô hình được thực hiện trong điều kiện tương đương.

Trong bài, Ridge Regression sử dụng hệ số regularization $\alpha = 1.0$. Biến mục tiêu được sử dụng ở log scale, tức là mô hình học trực tiếp trên log1p(SalePrice). Kết quả của mô hình baseline sẽ được sử dụng làm cơ sở đối chiếu với các mô hình MLP ở giai đoạn Evaluation.

5.2 Chuẩn bị dữ liệu cho PyTorch

Sau khi hoàn tất tiền xử lý, dữ liệu được chuyển sang dạng tensor kiểu float32 để có thể sử dụng trong PyTorch. Tập training gồm 1.168 mẫu với 248 đặc trưng, trong khi tập validation gồm 292 mẫu với 248 đặc trưng. Biến mục tiêu của MLP là log-price đã được chuẩn hóa và được biểu diễn dưới dạng một cột đầu ra.

Dữ liệu được tổ chức thành các batch thông qua DataLoader. Tập training được xáo trộn trong quá trình huấn luyện để hạn chế việc mô hình phụ thuộc vào thứ tự dữ liệu, trong khi tập validation không được shuffle. Seed của bộ sinh số ngẫu nhiên cũng được kiểm soát nhằm đảm bảo khả năng tái lập kết quả giữa các lần chạy.

Trong cấu hình hiện tại, `batch size` được đặt bằng 128. Ngoài ra, `split_seed = 42` được sử dụng để cố định cách chia train/validation, trong khi các `model_seeds` khác nhau được dùng để kiểm tra ảnh hưởng của khởi tạo trọng số và thứ tự batch đến kết quả mô hình.

5.3 Kiến trúc mạng MLP

Mô hình học sâu chính của bài thực hành là Multi-Layer Perceptron (MLP) được xây dựng bằng PyTorch. Sau quá trình tiền xử lý, mạng nhận 248 đặc trưng đầu vào và tạo ra một giá trị đầu ra duy nhất đại diện cho biến mục tiêu log-price đã được chuẩn hóa.

Các lớp ẩn của mạng sử dụng lớp kết nối đầy đủ Linear kết hợp với hàm kích hoạt ReLU. ReLU được lựa chọn để đưa tính phi tuyến vào mô hình, giúp mạng có khả năng học các mối quan hệ phức tạp hơn giữa các đặc trưng của căn nhà và giá bán.

Ngoài ra, Dropout với tỷ lệ 0,1 được áp dụng tại một số lớp ẩn nhằm hỗ trợ regularization và hạn chế hiện tượng overfitting. Lớp đầu ra chỉ gồm một neuron và không sử dụng hàm kích hoạt, phù hợp với bản chất của bài toán hồi quy.

MLP baseline có cấu trúc:

$$248 \to 128 \to 64 \to 32 \to 1$$

Trong đó 248 là số đặc trưng đầu vào sau tiền xử lý, ba lớp ẩn lần lượt có 128, 64 và 32 neuron, còn lớp cuối cùng có một neuron để tạo giá trị dự đoán.

5.4 Cấu hình huấn luyện

Quá trình huấn luyện MLP sử dụng Mean Squared Error (MSE) làm hàm mất mát. Đây là hàm loss phù hợp với bài toán hồi quy và cũng có mối liên hệ trực tiếp với RMSE được sử dụng trong quá trình đánh giá.

Thuật toán tối ưu được sử dụng là Adam, với learning rate bằng 0,001. Ngoài ra, tham số `weight_decay` = 10^{-5} được sử dụng như một hình thức regularization bổ sung nhằm hạn chế các trọng số của mạng tăng quá lớn.

Bảng 5.1: Các tham số huấn luyện MLP

Tham số	**Giá trị**
Batch size	128
Learning rate	0,001
Weight decay	10^{-5}
Max epochs	120
Patience	12
Dropout	0,1
Split seed	42

MSELoss được sử dụng làm hàm mất mát và Adam được sử dụng làm optimizer. Early Stopping dừng quá trình huấn luyện nếu validation loss không cải thiện trong 12 epoch liên tiếp và khôi phục trọng số ở best epoch.

5.5 Thử nghiệm nhiều kiến trúc

Để khảo sát ảnh hưởng của số lượng neuron trong các lớp ẩn, bài thực hành tiến hành huấn luyện ba cấu hình mạng:

$64 \to 32 \to 16$

$128 \to 64 \to 32$

$256 \to 128 \to 64$

Các kiến trúc được huấn luyện trên cùng training set và được đánh giá trên cùng validation set với seed cố định là 42. Việc cố định dữ liệu và seed ở bước này giúp đảm bảo sự khác biệt về kết quả chủ yếu đến từ cấu trúc của mạng thay vì thay đổi dữ liệu đánh giá.

Sau khi so sánh các kiến trúc, cấu hình có RMSE validation tốt hơn sẽ được chọn làm ứng viên cho bước kiểm tra độ ổn định ở nhiều seed. Ba kiến trúc trên và bộ seed 42, 123, 2026 đều đã được xác định ngay từ phần cấu hình thí nghiệm của notebook.

CHƯƠNG 6: EVALUATION

6.1 So sánh Ridge và MLP baseline

Đầu tiên, mô hình Ridge Regression được so sánh với MLP baseline có kiến trúc 128–64–32. Hai mô hình sử dụng cùng tập dữ liệu đã tiền xử lý và được đánh giá trên cùng validation set.

Bảng 6.1: So sánh Ridge Regression và MLP baseline

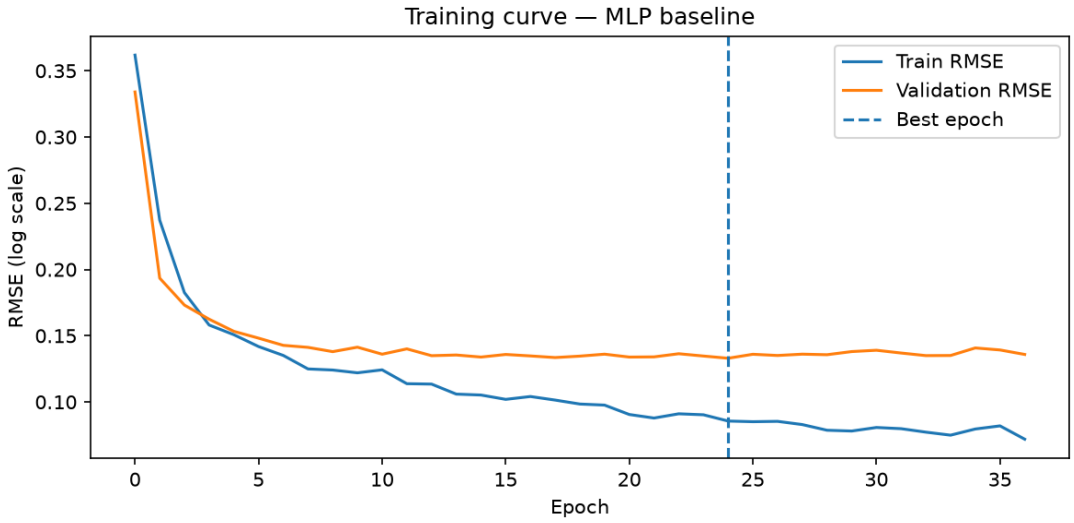
Mô hình	RMSE log	MAE (USD)	R^2
Ridge Regression	**0,1263**	**15.665,69**	**0,9261**
MLP 128–64–32	0,1330	15.977,49	0,8823

Kết quả cho thấy Ridge Regression đạt RMSE log thấp hơn MLP baseline, đồng thời có MAE thấp hơn và R² cao hơn. Cụ thể, Ridge đạt RMSE log 0,1263, trong khi MLP baseline đạt 0,1330. Điều này cho thấy với cấu hình MLP ban đầu, mô hình neural network chưa vượt qua mô hình hồi quy Ridge trên tập validation.

Kết quả này cũng cho thấy việc sử dụng Deep Learning không mặc nhiên đảm bảo chất lượng dự đoán tốt hơn đối với dữ liệu dạng bảng. Do đó, cần tiếp tục thử nghiệm nhiều kiến trúc MLP khác nhau trước khi đưa ra kết luận cuối cùng.

6.2 Đường cong huấn luyện

Trong quá trình huấn luyện MLP baseline, RMSE trên training set và validation set được theo dõi theo từng epoch nhằm quan sát khả năng học của mô hình và phát hiện dấu hiệu overfitting.



Hình 6.1: Đường cong RMSE của MLP baseline trên training và validation

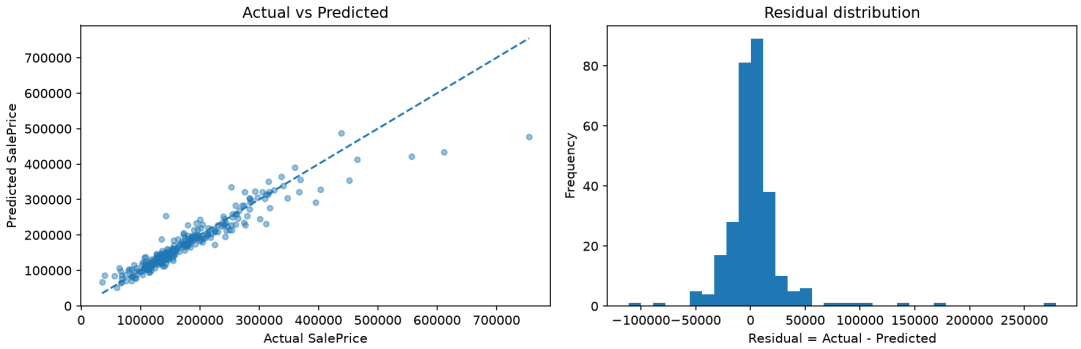
Thông qua đường cong huấn luyện có thể quan sát được RMSE giảm mạnh trong các epoch đầu, cho thấy mô hình nhanh chóng học được mối quan hệ giữa các đặc trưng đầu vào và biến mục tiêu. Sau một số epoch, validation RMSE không còn cải thiện rõ rệt, trong khi training RMSE vẫn có xu hướng giảm. Vì vậy, cơ chế Early Stopping được sử dụng để dừng quá trình huấn luyện và khôi phục trạng thái mô hình tại epoch tốt nhất.

Đối với MLP baseline, quá trình huấn luyện dừng tại epoch 37 và trọng số tại epoch

25 được lựa chọn là trạng thái tốt nhất. Mô hình này đạt RMSE log 0,1330, MAE khoảng 15.977,49 USD và R^2 0,8823.

6.3 Phân tích dự đoán và residual

Bên cạnh các chỉ số RMSE, MAE và R^2, khả năng dự đoán của MLP baseline còn được quan sát thông qua biểu đồ giữa giá thực tế và giá dự đoán, đồng thời phân tích phân phối residual.



Hình 6.2: So sánh SalePrice thực tế với giá dự đoán và phân phối residual

CTrong biểu đồ Actual vs Predicted, các điểm càng nằm gần đường chéo thì giá trị dự đoán càng gần với giá thực tế. Phần lớn các điểm có xu hướng phân bố quanh đường tham chiếu, cho thấy mô hình đã học được xu hướng chung của giá nhà. Tuy nhiên, vẫn tồn tại một số điểm nằm xa đường chéo, thể hiện những trường hợp mô hình dự đoán sai lệch đáng kể.

Biểu đồ residual biểu diễn phần chênh lệch giữa giá thực tế và giá dự đoán. Việc quan sát residual giúp đánh giá xem sai số của mô hình có tập trung quanh 0 hay không và phát hiện các trường hợp dự đoán bất thường. Notebook sử dụng cả hai biểu đồ này như một phần của quá trình đánh giá mô hình.

6.4 So sánh các kiến trúc MLP

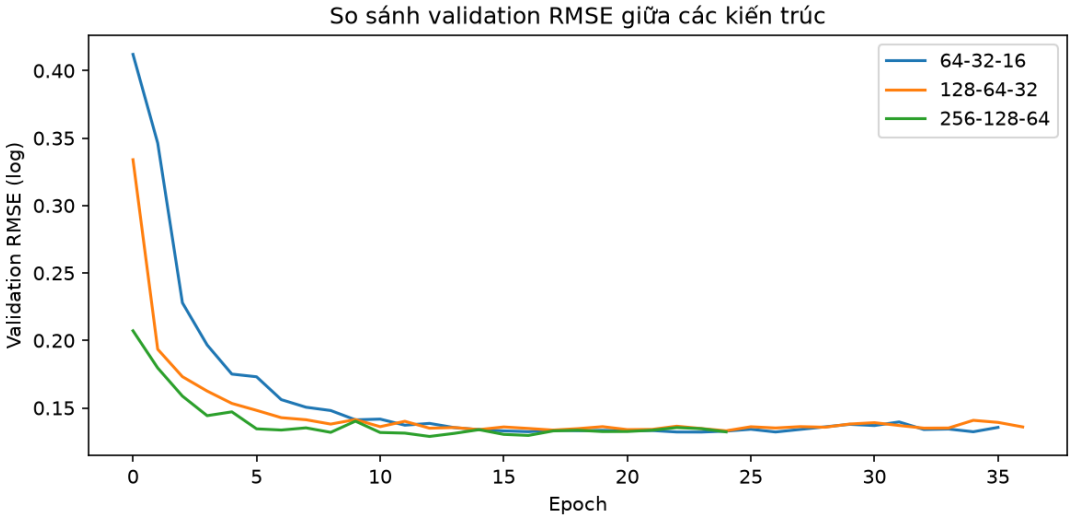
Sau MLP baseline, ba kiến trúc 64–32–16, 128–64–32 và 256–128–64 được huấn luyện trong cùng điều kiện với seed 42 để so sánh công bằng.

Bảng 6.2: Kết quả so sánh ba kiến trúc MLP với seed 42

Kiến trúc	Best epoch	RMSE log	MAE (USD)	R^2
256–128–64	**13**	**0,1288**	**15.249,02**	0,8951
64–32–16	24	0,1321	15.722,19	**0,8983**
128–64–32	25	0,1330	15.977,49	0,8823

Trong ba cấu hình được thử nghiệm, kiến trúc 256–128–64 đạt RMSE log thấp nhất là 0,1288, đồng thời có MAE thấp nhất là khoảng 15.249,02 USD. Vì RMSE log là metric chính của bài toán, kiến trúc này được lựa chọn để tiếp tục kiểm tra độ ổn định.

Mặc dù kiến trúc 64–32–16 có R^2 cao hơn một chút, việc lựa chọn mô hình được ưu tiên dựa trên RMSE log theo mục tiêu đã xác định từ đầu. Kết quả cũng cho thấy tăng số neuron không tạo ra sự cải thiện theo một quy luật hoàn toàn tuyến tính; hiệu quả còn phụ thuộc vào quá trình tối ưu và khả năng tổng quát hóa của mạng.



Hình 6.3: So sánh validation RMSE giữa các kiến trúc MLP

Biểu đồ cho phép quan sát sự thay đổi validation RMSE theo epoch của từng kiến trúc. Kiến trúc 256–128–64 đạt mức RMSE thấp hơn trong lần thử nghiệm với seed 42 và được chọn làm mô hình ứng viên cho bước kiểm tra tiếp theo.

6.5 Kiểm tra độ ổn định qua nhiều seed

Một hạn chế khi chỉ sử dụng một lần huấn luyện mạng neural là kết quả có thể chịu ảnh hưởng bởi các yếu tố ngẫu nhiên như khởi tạo trọng số và thứ tự batch. Vì vậy, kiến trúc 256–128–64 được chạy lại với ba seed khác nhau gồm 42, 123 và 2026, trong khi tập train/validation vẫn được giữ cố định.

Bảng 6.3: Kết quả stability test của kiến trúc 256–128–64

Seed	Best epoch	RMSE log	MAE (USD)	R^2
42	13	0,1288	15.249,02	0,8951
123	17	0,1309	15.943,94	0,9064
2026	8	0,1326	15.943,79	0,8965

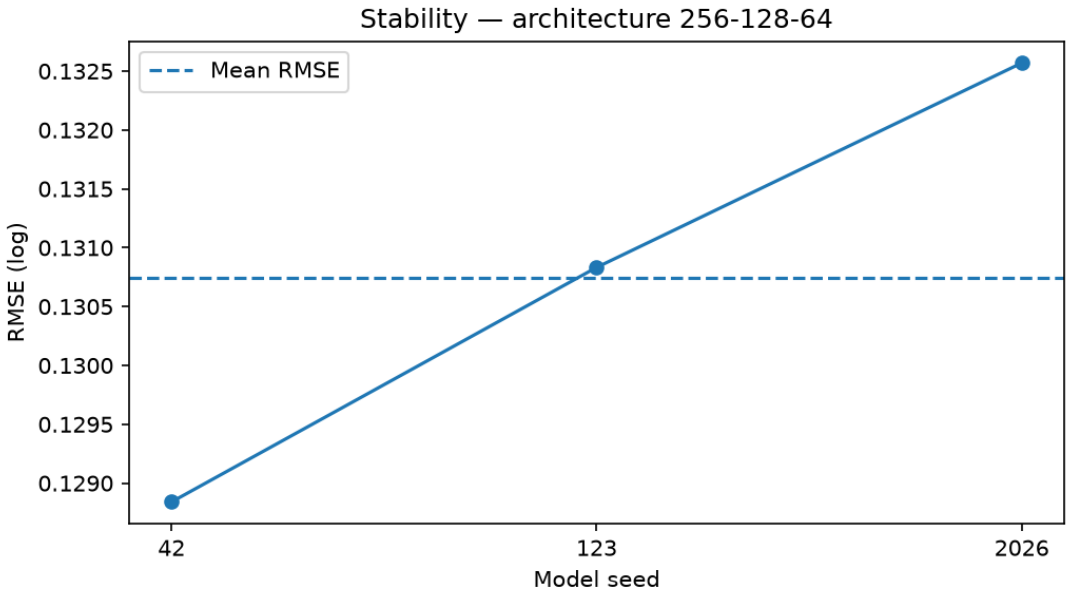
Kết quả cho thấy RMSE log dao động trong khoảng từ 0,1288 đến 0,1326. Sự chênh lệch giữa các lần chạy tương đối nhỏ, cho thấy kiến trúc được lựa chọn có mức độ ổn định khá tốt trước sự thay đổi của seed.

Bảng 6.4: Kết quả trung bình qua ba seed

Chỉ số	Giá trị
Mean RMSE log	0,1308
Std RMSE log	0,0019
Mean MAE	15.712,25 USD
Mean R^2	0,8994
Median best epoch	13

Độ lệch chuẩn RMSE chỉ khoảng 0,0019, cho thấy kết quả giữa các lần huấn luyện không biến động quá lớn. Vì vậy, kiến trúc 256–128–64 được lựa chọn làm kiến trúc

cuối cùng của mô hình MLP. Median của best epoch bằng 13 cũng được sử dụng làm cơ sở xác định số epoch cho giai đoạn huấn luyện mô hình cuối.



Hình 6.4: Sự thay đổi RMSE log của kiến trúc 256–128–64 theo seed

6.6 So sánh mô hình MLP cuối với Ridge Regression

Sau khi thử nghiệm kiến trúc và kiểm tra độ ổn định, MLP 256–128–64 đạt Mean RMSE log = 0,1308, trong khi Ridge Regression đạt RMSE log = 0,1263. Như vậy, Ridge vẫn có RMSE thấp hơn MLP trung bình khoảng 0,0044.

Kết quả này cần được nhìn nhận khách quan. MLP đã cải thiện so với cấu hình baseline 128–64–32 và đạt kết quả ổn định qua nhiều seed, tuy nhiên vẫn chưa vượt được Ridge Regression trên validation set. Điều này cho thấy với bộ dữ liệu House Prices có kích thước không quá lớn và thuộc dạng tabular, một mô hình hồi quy có regularization vẫn có thể đạt hiệu quả rất cạnh tranh.

Tuy nhiên, mục tiêu chính của bài thực hành là triển khai một mạng MLP bằng PyTorch, xây dựng quy trình huấn luyện, so sánh nhiều kiến trúc và đánh giá tính ổn định của mô hình. Theo tiêu chí này, mô hình MLP đã đáp ứng đầy đủ yêu cầu và cho kết quả dự đoán ở mức tốt.

6.7 Kết luận giai đoạn Evaluation

Qua quá trình đánh giá, kiến trúc 256–128–64 được lựa chọn là cấu hình MLP phù hợp nhất trong ba kiến trúc đã thử nghiệm. Mô hình đạt RMSE log trung bình 0,1308 ± 0,0019 qua ba seed, với MAE trung bình khoảng 15.712,25 USD và R^2 trung bình 0,8994. Kết quả tương đối ổn định giữa các lần chạy cho thấy kiến trúc này có khả năng tổng quát hóa khá tốt trên tập validation.

Mặc dù Ridge Regression vẫn đạt RMSE log thấp hơn, MLP cho thấy khả năng học tốt trên dữ liệu sau tiền xử lý và đáp ứng mục tiêu Deep Learning của bài thực hành. Từ kết quả này, kiến trúc 256–128–64 được sử dụng trong giai đoạn tiếp theo để huấn luyện lại trên toàn bộ tập train và dự đoán tập test.

CHƯƠNG 7: DEPLOYMENT VÀ TỔNG KẾT

7.1 Kết quả đạt được

Trong bài thực hành này, nhóm đã xây dựng hoàn chỉnh quy trình dự đoán giá nhà trên bộ dữ liệu House Prices: Advanced Regression Techniques theo các giai đoạn của CRISP-DM. Quá trình thực hiện bao gồm xác định bài toán, khám phá dữ liệu, xử lý dữ liệu thiếu, mã hóa các biến phân loại, chuẩn hóa dữ liệu, xây dựng mô hình, đánh giá và tạo kết quả dự đoán cho tập test. Notebook đã bao quát đầy đủ các giai đoạn Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation và Deployment.

Trong giai đoạn tiền xử lý, bài thực hành chú trọng hạn chế hiện tượng data leakage bằng cách chia tập train và validation trước khi học các tham số tiền xử lý. Các thống kê dùng để điền dữ liệu thiếu, mã hóa và chuẩn hóa trong quá trình đánh giá chỉ được xác định trên training fold. Biến mục tiêu SalePrice được biến đổi bằng log1p nhằm giảm độ lệch của phân bố và tạo điều kiện thuận lợi hơn cho quá trình huấn luyện mạng neural.

Hai nhóm mô hình được sử dụng gồm Ridge Regression làm baseline và Multi-Layer Perceptron bằng PyTorch làm mô hình Deep Learning chính. Đối với MLP, ba kiến trúc 64–32–16, 128–64–32 và 256–128–64 được thử nghiệm trong cùng điều kiện. Kết quả cho thấy kiến trúc 256–128–64 đạt RMSE log tốt nhất trong số các cấu hình MLP được khảo sát và được lựa chọn để tiếp tục kiểm tra độ ổn định.

Khi chạy kiến trúc 256–128–64 với các seed 42, 123 và 2026, mô hình đạt RMSE log trung bình 0,1308, độ lệch chuẩn 0,0019, MAE trung bình khoảng 15.712,25 USD và R^2 trung bình 0,8994. Độ lệch chuẩn RMSE tương đối nhỏ cho thấy kết quả của mạng không thay đổi quá lớn giữa các lần huấn luyện. Median của best epoch là 13, vì vậy 13 epoch được lựa chọn để huấn luyện mô hình cuối cùng.

Sau khi hoàn thành quá trình lựa chọn kiến trúc và số epoch, pipeline tiền xử lý được fit lại trên toàn bộ 1.460 mẫu huấn luyện nhằm tận dụng tối đa dữ liệu có sẵn. Mô hình MLP cuối cùng sau đó được huấn luyện trong 13 epoch và sử dụng để dự đoán giá bán cho 1.459 căn nhà trong tập test. Trước khi xuất kết quả, phân bố giá dự đoán được kiểm tra nhằm phát hiện các giá trị bất thường. Giá dự đoán có trung vị khoảng 154.423 USD, trung bình khoảng 177.152 USD, giá trị nhỏ nhất khoảng 44.621 USD và lớn nhất khoảng 576.635 USD, nằm trong phạm vi hợp lý khi so sánh với dữ liệu train.u trên tập validation sau khi đã hoàn tất quá trình lựa chọn mô hình.

7.2 Nhận xét về kết quả mô hình

Một kết quả đáng chú ý của bài thực hành là Ridge Regression đạt RMSE log 0,1263, thấp hơn RMSE log trung bình của MLP là 0,1308 khoảng 0,0044. Do RMSE càng thấp càng tốt, trong thí nghiệm hiện tại Ridge Regression đạt kết quả validation tốt hơn MLP.

Kết quả này cho thấy mạng neural không phải lúc nào cũng vượt trội hơn các phương pháp học máy truyền thống, đặc biệt đối với dữ liệu dạng bảng có số lượng mẫu tương đối nhỏ. Ridge Regression có cấu trúc đơn giản và regularization phù hợp nên vẫn có khả năng đạt kết quả cạnh tranh trên bài toán này.

Tuy nhiên, mục tiêu trọng tâm của bài Lab là xây dựng và đánh giá một mô hình MLP bằng PyTorch. Theo mục tiêu đó, bài thực hành đã hoàn thành được toàn bộ quy trình từ chuẩn bị tensor, xây dựng kiến trúc mạng, sử dụng hàm mất mát MSE và Adam,

áp dụng Dropout và Early Stopping, so sánh nhiều kiến trúc đến đánh giá độ ổn định qua nhiều seed. Việc báo cáo cả kết quả của Ridge và MLP cũng giúp quá trình đánh giá khách quan hơn thay vì chỉ trình bày mô hình Deep Learning.

7.3 Hạn chế và hướng phát triển

Mặc dù đã đạt được kết quả tương đối ổn định, bài thực hành vẫn còn một số hướng có thể tiếp tục cải thiện. Bộ dữ liệu House Prices có số lượng mẫu không lớn so với số đặc trưng sau khi mã hóa, vì vậy MLP có thể chưa phát huy tối đa ưu thế của mạng neural. Bên cạnh đó, bài thực hành mới khảo sát ba kiến trúc mạng với một số hyperparameter cố định nên chưa thể khẳng định đây là cấu hình MLP tối ưu tuyệt đối.

Trong các nghiên cứu tiếp theo, có thể mở rộng việc tìm kiếm hyperparameter như số lớp ẩn, số neuron, learning rate, batch size, dropout và weight decay; đồng thời sử dụng các phương pháp đánh giá như K-Fold Cross Validation để giảm mức phụ thuộc vào một lần chia train/validation. Ngoài ra, có thể xây dựng thêm các đặc trưng mới từ dữ liệu gốc hoặc so sánh MLP với các mô hình phù hợp cho dữ liệu tabular như Random Forest, Gradient Boosting hoặc XGBoost để có cái nhìn toàn diện hơn về hiệu quả của từng phương pháp.

Nhìn chung, bài thực hành đã xây dựng thành công một quy trình dự đoán giá nhà có kiểm soát data leakage, triển khai được mạng MLP bằng PyTorch, đánh giá được hiệu quả và độ ổn định của mô hình, đồng thời tạo được file dự đoán cuối cùng phục vụ cho việc nộp kết quả. Qua đó, bài thực hành giúp củng cố kiến thức về xử lý dữ liệu dạng bảng, xây dựng mạng neural cho bài toán hồi quy và đánh giá mô hình một cách có hệ thống.

TÀI LIỆU THAM KHẢO

[1] D. De Cock, "Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project," *Journal of Statistics Education*, vol. 19, no. 3, 2011.
https://jse.amstat.org/v19n3/decock.pdf

[2] Kaggle, "House Prices: Advanced Regression Techniques."
https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques

[3] Rishabh Nimje, *House Prices: Advanced Regression Techniques*, tài liệu/source tham khảo được cung cấp trong bài thực hành.

[4] PyTorch, "PyTorch Documentation."
https://pytorch.org/docs/stable/index.html

[5] Scikit-learn Developers, "Scikit-learn Documentation."
https://scikit-learn.org/stable/